C++

程式語言（二）實習課

Introduction to Programming (II) Practice Lessons

Course Introduction

Joseph Chuang-Chieh Lin

Dept. CSE, NTOU

Platform

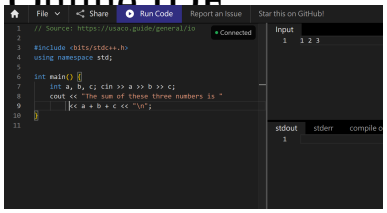
- Dev-C++

  Click here to download.

  Note: Please use this version otherwise you can't compile your programs/projects in Win10.

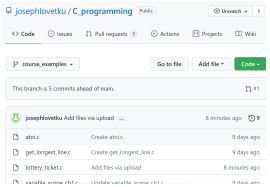
- OnlineGDB (https://www.onlinegdb.com/)

- Real-Time Collaborative Online IDE (https://ide.usaco.guide/)



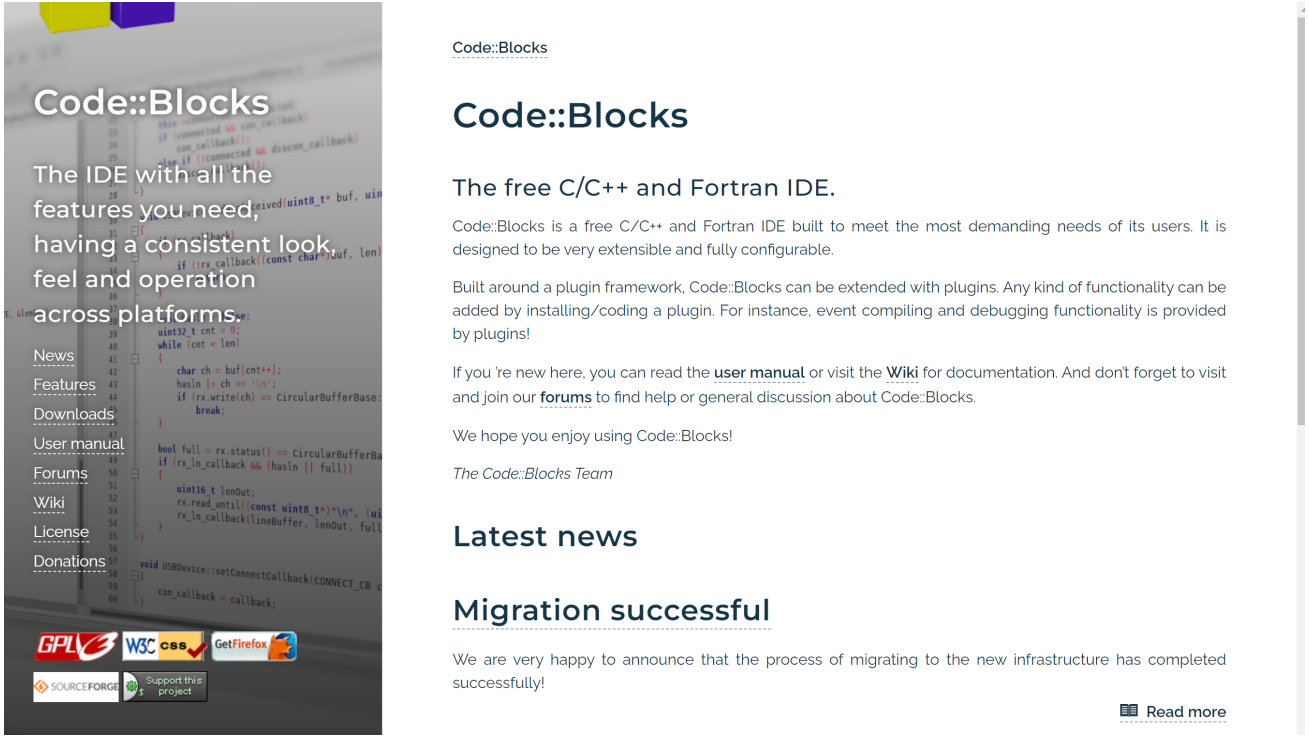
- Other resources:
  - MIT OpenCourseWare - Introduction to C++ [link].
  - Learning C++ Programming [Programiz].
  - GeeksforGeeks [link]

My GitHub page:
click the link here to visit.



Platform/IDE

- https://www.codeblocks.org/



Textbooks (We focusing on C++11)

- ***Learn C++ Programming by Refactoring* (由重構學習 C++ 程式設計). Pang-Feng Liu (劉邦鋒). NTU Press. 2023.**
- ***C++ Primer. 5th Edition.* Stanley B. Lippman, Josée Lajoie, Barbara E. Moo. 2019.**
- *Effective C++*. Scott Meyers. O'Reilly. 2016.
- *Thinking in C++. Vol. 1: Introducing to Standard C++*. 2nd Edition. Bruce Eckel. Prentice Hall PTR. 2000.

Useful Resources

- Tutorialspoint
  - https://www.tutorialspoint.com/cplusplus/index.htm
  - Online C++ Compiler
- Programiz
  - https://www.programiz.com/cpp-programming

Course TAs

- 林琮祐
  - Email address: 11357051@o365.ntou.edu.tw
- 張意苹
  - Email address: 11457070@o365.ntou.edu.tw
- 楊昀潼
  - Email address: 11457074@o365.ntou.edu.tw

Grading Policy

- Assignments: 60%
- Midterm Exam: 20%
- Final Exam: 20%

Notes on Homework Submission

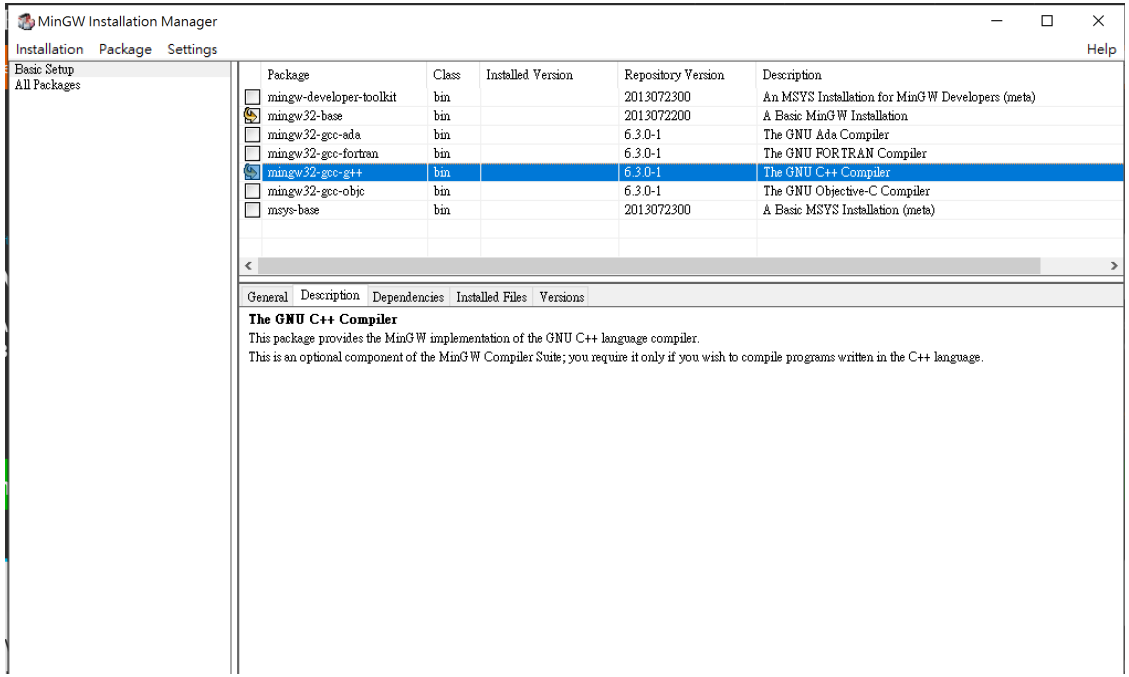
- Makeup due date:
  - 1 week after the due date.
  - 40% discount.

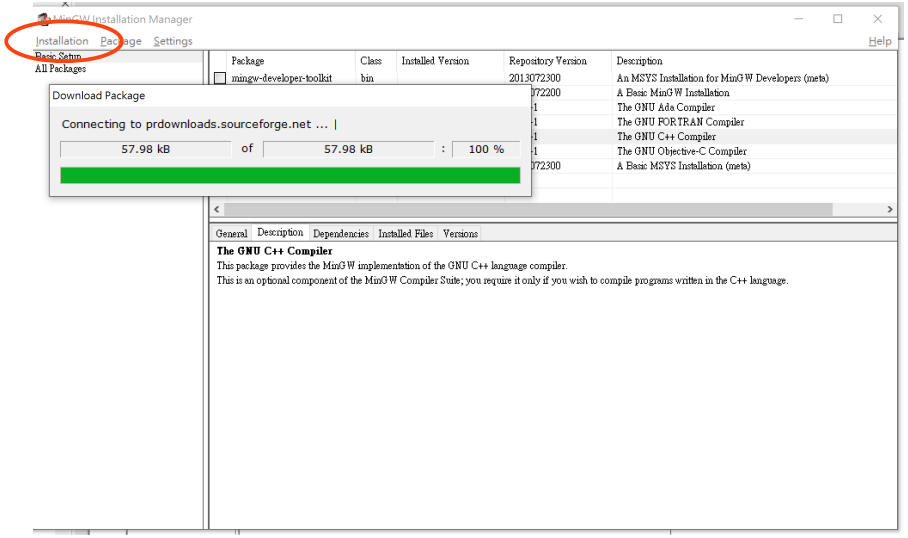
Appendix:

Project Compilation

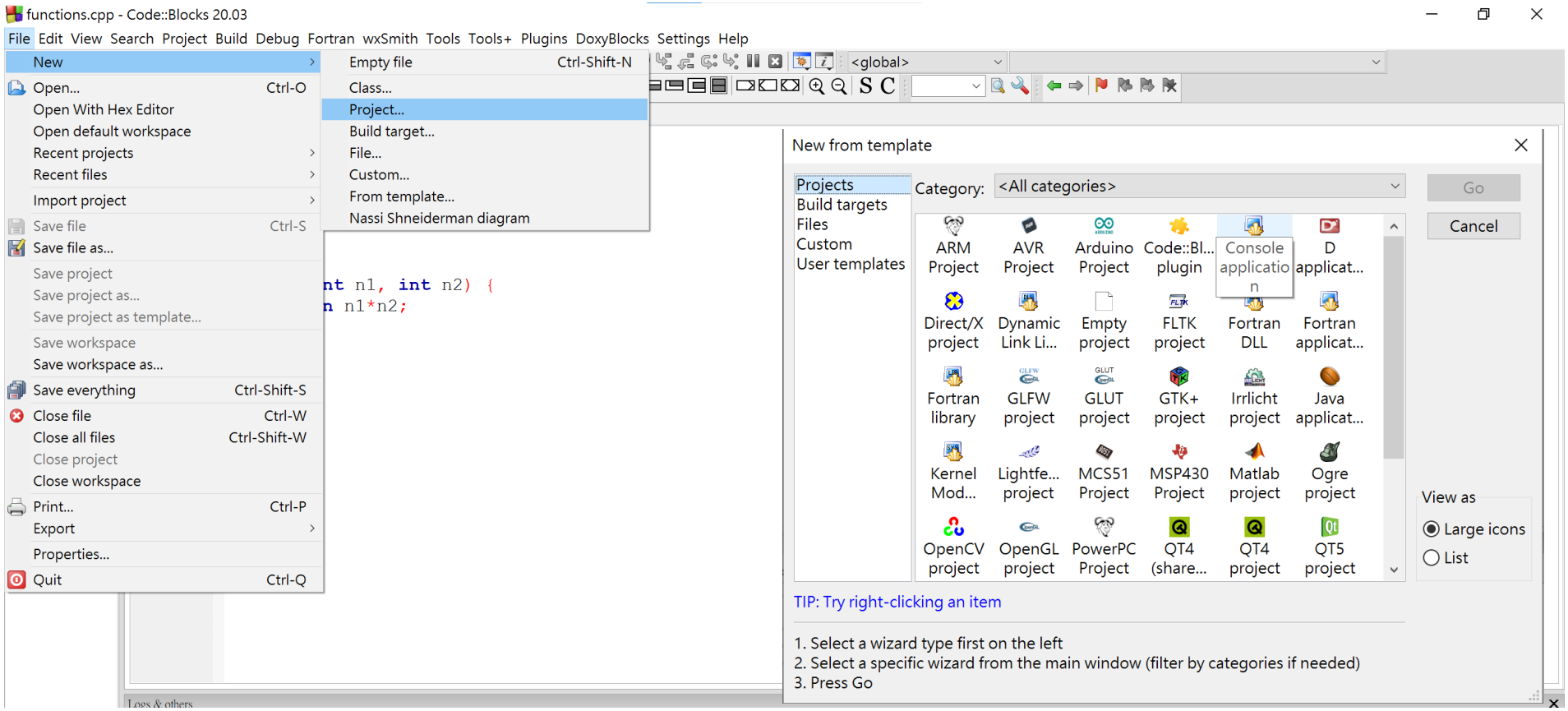
MinGW Installation

https://sourceforge.net/projects/mingw/files/latest/download

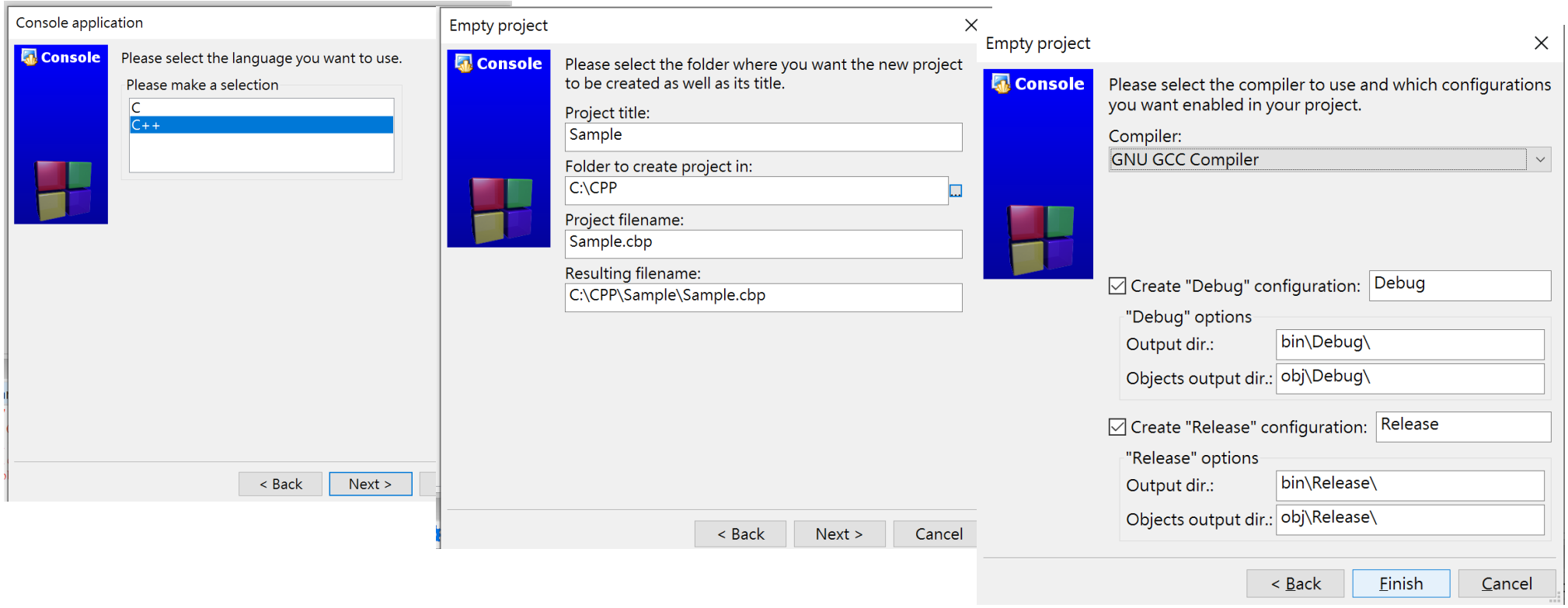




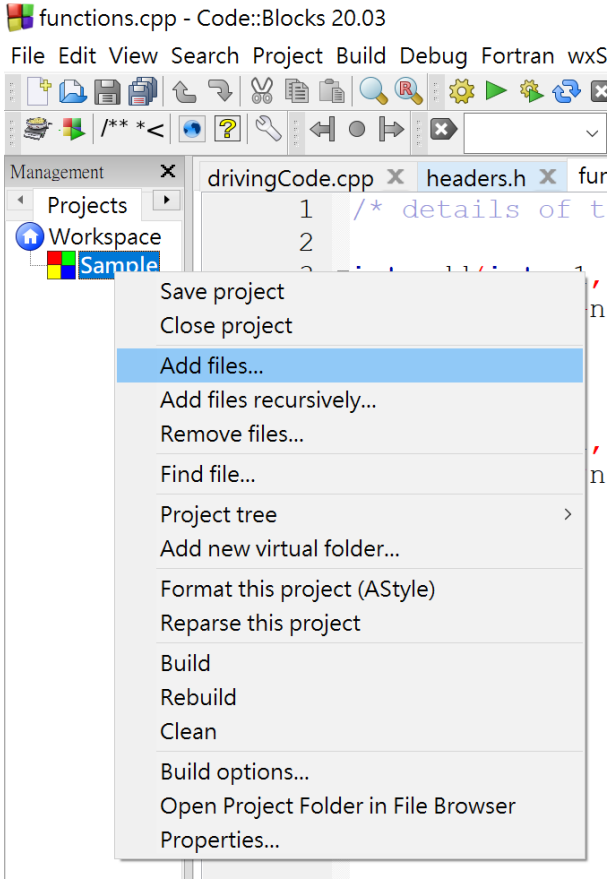
Codeblocks

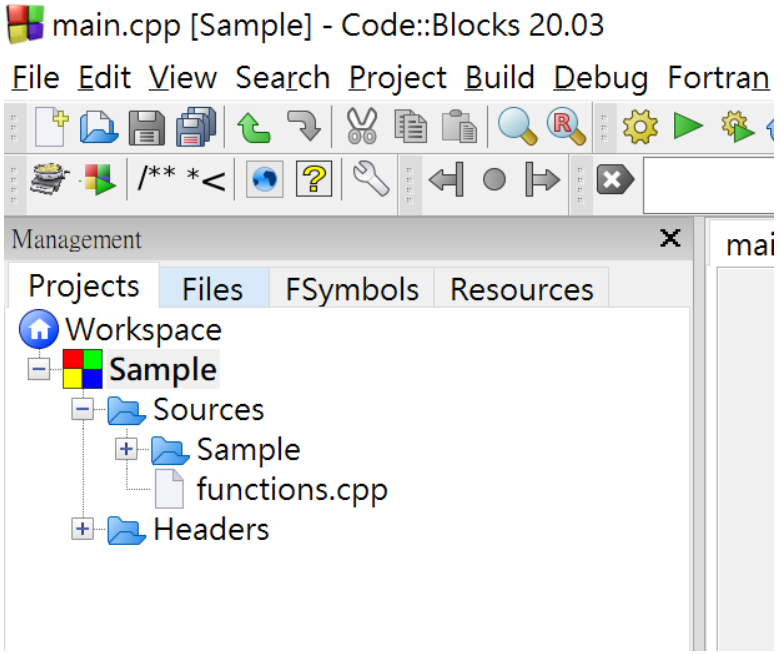

functions.cpp - Code::Blocks 20.03
File Edit View Search Project Build Debug Fortran wxSmith Tools Tools+ Plugins DoxyBlocks Settings Help
New
Open... Ctrl-O
Open With Hex Editor
Open default workspace
Recent projects
Recent files
Import project
Save file Ctrl-S
Save file as...
Save project
Save project as...
Save project as template...
Save workspace
Save workspace as...
Save everything Ctrl-Shift-S
Close file Ctrl-W
Close all files Ctrl-Shift-W
Close project
Close workspace
Print... Ctrl-P
Export
Properties...
Quit Ctrl-Q
Empty file Ctrl-Shift-N
Class...
Project...
Build target...
File...
Custom...
From template...
Nassi Shneiderman diagram
New from template
Projects
Build targets
Files
Custom
User templates
Category: <All categories>
Go
Cancel
ARM Project
AVR Project
Arduino Project
Code::Bl... plugin
Console application
D applicat...
Direct/X project
Dynamic Link Li...
Empty project
FLTK project
Fortran DLL
Fortran applicat...
Fortran library
GLFW project
GLUT project
GTK+ project
Irrlicht project
Java applicat...
Kernel Mod...
Lightfe... project
MCS51 Project
MSP430 Project
Matlab project
Ogre project
OpenCV project
OpenGL project
PowerPC Project
QT4 (share...
QT4 project
QT5 project
View as
Large icons
List
TIP: Try right-clicking an item
1. Select a wizard type first on the left
2. Select a specific wizard from the main window (filter by categories if needed)
3. Press Go

Steps...


Console application
Console
Please select the language you want to use.
Please make a selection
C
C++
< Back
Next >
Empty project
Console
Please select the folder where you want the new project to be created as well as its title.
Project title:
Sample
Folder to create project in:
C:\CPP
Project filename:
Sample.cbp
Resulting filename:
C:\CPP\Sample\Sample.cbp
< Back
Next >
Cancel
Empty project
Console
Please select the compiler to use and which configurations you want enabled in your project.
Compiler:
GNU GCC Compiler
Create "Debug" configuration:
Debug
"Debug" options
Output dir.:
bin\Debug\
Objects output dir.:
obj\Debug\
Create "Release" configuration:
Release
"Release" options
Output dir.:
bin\Release\
Objects output dir.:
obj\Release\
< Back
Finish
Cancel

Adding project files...





main.cpp

main.cpp

```cpp
#include <iostream>
#include "headers.h"

using namespace std;

int main() {
    int num1, num2;
    cin >> num1 >> num2;
    cout << add(num1, num2) << endl;
    cout << mul(num1, num2) << endl;
    return 0;
}
```

functions.cpp

main.cpp functions.cpp

```cpp
/* details of the functions */

int add(int n1, int n2) {
    return n1+n2;
}


int mul(int n1, int n2) {
    return n1*n2;
}

```

headers.h

main.cpp | functions.cpp | headers.h

```
1  #ifndef _HEADERS_H_
2  #define _HEADERS_H_
3
4  int add(int n1, int n2);
5  int mul(int n1, int n2);
6
7  #endif
8
```

In the hindsight

命令提示字元

```
C:\CPP\Sample>g++ -c functions.cpp -o functions.o

C:\CPP\Sample>g++ -c main.cpp -o main.o

C:\CPP\Sample>g++ functions.o main.o -o sample.exe

C:\CPP\Sample>sample.exe
12 20
32
240

C:\CPP\Sample>
```

Online GDB https://www.onlinegdb.com/#

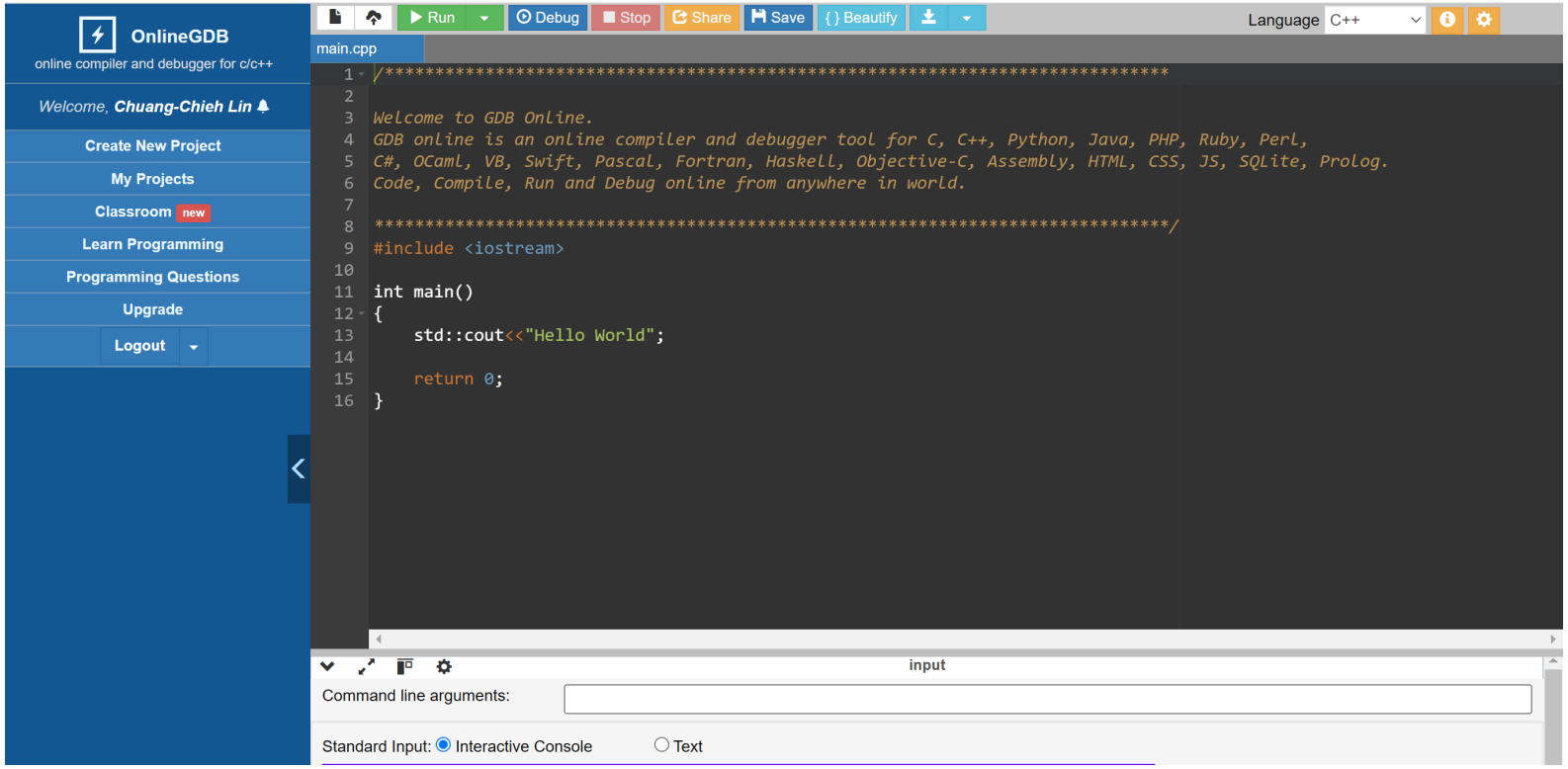


Illustration (1/2)

OnlineGDB
online compiler and debugger for c/c++

Welcome, **Chuang-Chieh Lin**

Sample "Project"
Create New Project
My Projects
Classroom new
Learn Programming
Programming Questions
Upgrade
Logout

Run | Debug | Stop | Share | Saved | { } Beautify | Language C++

main.cpp | functions.cpp | headers.h

```
/*****************************************************************************

A Very Simple Sample Project in OnlineGDB
Author: Joseph Chuang-Chieh Lin
Date: Feb. 15, 2025

*****************************************************************************/
#include <iostream>
#include "headers.h"

using namespace std;

int main() {
    int num1, num2;
    cin >> num1 >> num2;
    cout << add(num1, num2) << endl;
    cout << mul(num1, num2) << endl;
    return 0;
}
```

input

```
12 20
32
240

...Program finished with exit code 0
Press ENTER to exit console.
```

About • FAQ • Blog • Terms of Use • Contact Us • GDB Tutorial • Credits • Privacy
© 2016 - 2025 GDB Online

Illustration (2/2)

OnlineGDB
online compiler and debugger for c/c++

Welcome, **Chuang-Chieh Lin**

Sample "Project"
Create New Project
My Projects
Classroom new
Learn Programming
Programming Questions
Upgrade
Logout

Run | Debug | Stop | Share | Save | { } Beautify | Language C++

main.cpp | functions.cpp | headers.h

```cpp
/*****************************************************************************

A Very Simple Sample Project in OnlineGDB
Author: Joseph Chuang-Chieh Lin
Date: Feb. 15, 2025

*****************************************************************************/
#include <iostream>
#include "headers.h"

using namespace std;

int main() {
    int num1, num2;
    cin >> num1 >> num2;
    cout << add(num1, num2) << endl;
    cout << mul(num1, num2) << endl;
    return 0;
}
```

input

Command line arguments:

Standard Input: Interactive Console Text

12 20

About • FAQ • Blog • Terms of Use • Contact Us • GDB Tutorial • Credits • Privacy
© 2016 - 2025 GDB Online

Illustration (2/2)

OnlineGDB
online compiler and debugger for c/c++

Welcome, **Chuang-Chieh Lin**

Sample "Project"

Create New Project
My Projects
Classroom new
Learn Programming
Programming Questions
Upgrade
Logout

Run | Debug | Stop | Share | Save | { } Beautify | Language C++

main.cpp | functions.cpp | headers.h

```cpp
/*****************************************************************************

A Very Simple Sample Project in OnlineGDB
Author: Joseph Chuang-Chieh Lin
Date: Feb. 15, 2025

*****************************************************************************/
#include <iostream>
#include "headers.h"

using namespace std;

int main() {
    int num1, num2;
    cin >> num1 >> num2;
    cout << add(num1, num2) << endl;
    cout << mul(num1, num2) << endl;
    return 0;
}
```

input | stdout | stderr

Compiled Successfully. memory: 3584 time: 0 exit code: 0

```
32
240
```

About • FAQ • Blog • Terms of Use • Contact Us • GDB Tutorial • Credits • Privacy
© 2016 - 2025 GDB Online

Introduction to OOP

Object-Oriented Programming Languages

- [Wikipedia] A programming paradigm based on the concept of "objects", which can contain data and code.
  - Data: in the form of fields (i.e., *attributes* or *properties*)
  - Codes: procedures (i.e., *methods*).
- Examples:
  - C++, JAVA, Python, Perl, Lisp.

- Most popular: *Class*-based.
  - Objects are instances of classes.

C++

- Created by Bjarne Stroustrup (a Danish computer scientist; C++ started in 1979 and has become generally available since 1985).
- Originally C with classes and renamed as C++.
- Efficiency of C is maintained.
- Applications in
  - System software
  - Application software
  - Device drivers
  - Embedded software
  - Games
  - High-performance computing server
  - Used to program FPGAs (Field Programmable Gate Arrays)

Why C++?

- Roughly a superset of C.
- Maintainability and Portability.
- International standard(s)
  - “C with Classes” -> C++03 -> C++11 -> C++14 -> C++17 -> C++20.
- General purpose.
- Powerful yet efficient.
- Low-level access to hardware.
- Easy to move to other OOP languages
  - But NOT in other direction.

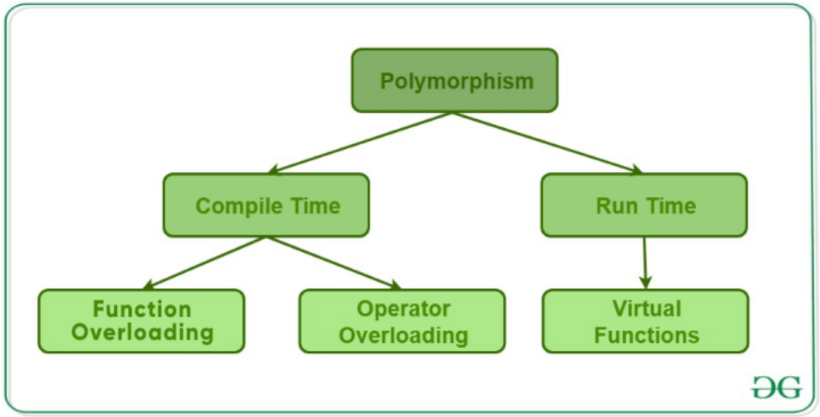
The four basic concepts of OOP

- **Encapsulation (封裝)**
  - **Wrapping up** of data and information under a single unit.
  - Binding together the data and the functions that manipulates them.
  - Emphasize on the "**interface**".
  - Using classes.
  - Protect the data from outside interference and misuse.

- **Abstraction (抽象化)**
  - Displaying only **essential information** and hiding the details.
  - Providing only essential information about the data to the outside world.
  - **Hiding the details and implementation.**
  - Using classes or header files.
  - Reducing complexity and allowing the programmer to focus on interactions at a higher level.

The four basic concepts of OOP

- **Inheritance (繼承)**
  - The ability of **deriving properties and characteristics from another class**.
  - Promoting code **reusability** and establishing a natural **hierarchy** between classes.
- **Polymorphism (多型)**
  - The ability of a message to be displayed **in more than one form**.



Reference:
https://www.geeksforgeeks.org/polymorphism-in-c/?ref=lbp

GNU Compiler Collection (GCC)
C++ Compiler

```
g++ [options] input_file.cpp
```

To object code file:

```
g++ -c file_1.cpp // get file_1.o
g++ -c file_2.cpp // get file_2.o
```

To link object code files and produce a executable file:

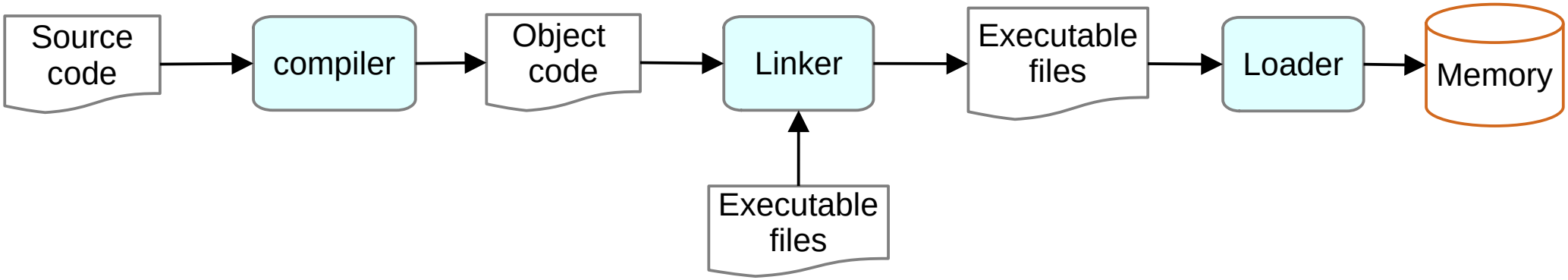
```
g++ -o executable.exe file_1.o file_2.o
```

To directly produce a executable file from a C++ source code:

```
g++ -o executable.exe source.cpp
```

Flowchart

Linker: combines the object files (e.g., generated by the compiler/assembler) and generate executable files.



Loader: load executable files to main memory.

Your first C++ program

```
#include <iostream>

int main() {
    std::cout << "Welcome to C++!\n"; // :: scope resolution operator
    return 0;
}
```

```
#include <iostream>
using namespace std;

int main() {
    cout << "Welcome to C++!\n";
    //cout << "Welcome to C++!" << endl;
    return 0;
}
```

A simple I/O example

```
#include <iostream>
using namespace std;

int main() {
    int n1 = 0, n2 = 0;
    cout << "Enter two numbers: " << endl;
    cin >> n1 >> n2;
    cout << "The sum is " << n1+n2 << endl;

    return 0;
}
```

A typical syntax error (語法錯誤)

```
#include <iostream>
using namespace std;

int main() {
    int n1 = 0, n2 = 0;
    cin >> a >> b >> c;
    cout << a * b / c << endl

    return 0;
}
```

What's wrong with the above example?

main() with no return type

```
#include <iostream>

using namespace std;

main()
{
    cout << "Hello World";
}
```

```
#include <iostream>

using namespace std;

void main()
{
    cout << "Hello World";
}
```

Continually getting input numbers

```
#include <iostream>
using namespace std;

int main() {
    int sum = 0, value = 0;
    // continually getting numbers and sum over them until
    // reaching end-of-file
    while (cin >> value)
        sum += value;
    cout << "The sum is " << sum << endl;

    return 0;
}
```

Note:
In Windows OS, use `Ctrl+z` to represent end-of-file.
In Mac OS or Unix/Linux OS, use `Ctrl+d` to represent end-of-file.

String I/O

```
#include <iostream>
using namespace std;

int main()
{
    char str[100];

    cout << "Enter a string: ";
    cin >> str;
    cout << "You entered: " << str << endl;

    return 0;
}
```

String I/O: read a line of text

```
#include <iostream>
#include <string>
using namespace std;

int main()
{
    char str[100];
    //string str;

    cout << "Enter a string: ";
    cin.get(str, 100);
    //getline(cin, str);
    cout << "You entered: " << str << endl;

    return 0;
}
```

Supplementary

- Difference between

  ```
  cout << "blah blah ..." << endl;
  ```

  and

  ```
  cout << "blah blah ..." << "\n";
  ```

- `cout << "blah blah ..." << endl;`

  is equivalent to

  `cout << "blah blah ..." << "\n"` **and then flush the buffer.**